

call me maybe

Introducción a las llamadas a función en LLMs

Resumen: ¿Pueden los LLM hablar el lenguaje de los ordenadores? Lo descubriremos.
Hecho en colaboración con @ldevelle, @pcamaren, @crfernan

Versión: 1.0

Índice general

I. Prólogo **2**

II. Instrucciones sobre la IA **3**

III. Introducción **6**
III.1. ¿Qué es la llamada a función? 6
III.2. ¿Por qué es importante? 7
III.3. El desafío 7

IV. Common Instructions **8**
IV.1. General Rules 8
IV.2. Makefile 8
IV.3. Additional Guidelines 9
IV.3.1. Requisitos adicionales 9
IV.3.2. Uso 10

V. Parte obligatoria **11**
V.1. Resumen 11
V.2. Archivos de entrada 11
V.2.1. Pruebas de llamadas a funciones 11
V.2.2. Definiciones de funciones 12
V.3. Interacción con el LLM 13
V.3.1. El SDK del LLM 13
V.3.2. El proceso de generación 13
V.3.3. Comprender la decodificación restringida 15
V.4. Formato del archivo de salida 16
V.4.1. Ejemplo de salida 16
V.4.2. Reglas de validación 16
V.5. Rendimiento y fiabilidad 17
V.6. Probar tu implementación 17

VI. Requisitos del Readme **18**

VII. Entrega y evaluación entre pares **20**
VII.1. Instrucciones de recodificación 20

Capítulo I

Prólogo

En la antigua roma se tallaban tabletas de piedra con cuadrículas perfectas para seguir los envíos de grano por todo el imperio, de modo que ningún envío se perdiera por culpa de una mala caligrafía. En el siglo XIX, los marineros registraban las corrientes oceánicas en tablas estructuradas tan precisas que algunas todavía se usan hoy en navegación.

Los primeros pronósticos meteorológicos se enviaban como telegramas en un código fijo: unos pocos números que podían describir todo un cielo. En los años 60, el control de misión de la NASA trabajaba con diagramas de flujo plastificados que indicaban exactamente qué significaba cada luz intermitente, independientemente de quién estuviera de turno. En los supermercados, los escáneres de códigos de barras traducían franjas en blanco y negro en datos de inventario mucho antes de que la mayoría de los hogares tuviera ordenadores. Incluso las personas apicultoras han usado formularios estandarizados durante décadas, registrando el estado de las colmenas, el flujo de néctar y el linaje de la reina en pequeñas casillas que solo ellas podían leer de un vistazo.

Las personas siempre hemos construido estructuras para que la información sea fiable, compartible y utilizable. Y eso nos lleva al presente, donde el objetivo es hacer que la IA hable el lenguaje de los ordenadores.

Capítulo II

Instrucciones sobre la IA

Contexto

Durante tu proceso de aprendizaje, la IA puede ayudarte con muchas tareas diferentes. Tómate el tiempo necesario para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyarte con tu trabajo. Sin embargo, siempre debes abordarlas con precaución y evaluar de forma crítica los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca podrás saber con total certeza si tu pregunta está bien formulada o si el contenido generado es el adecuado. Las personas que te rodean son tu recurso más valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal:

- Utiliza la IA para reducir las tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting, ya sea para programacion o para otros temas, que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar de forma eficiente y evitar los riesgos comunes, sesgos y problemas éticos.
- Sigue trabajando con tus compañeros para desarrollar tanto habilidades técnicas como habilidades transversales.
- Utiliza únicamente contenido generado por IA que entiendas completamente y del cual puedas responsabilizarte.

Reglas para estudiantes:

- Debes tomarte el tiempo necesario para explorar las herramientas de IA y comprender cómo funcionan, para poder utilizarlas de manera ética y reducir los sesgos potenciales.
- Debes reflexionar sobre tu problema antes de dar instrucciones a la IA. Esto te ayuda a escribir preguntas, instrucciones o conjuntos de datos más claros, detalladas y relevantes utilizando un vocabulario preciso.

- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier contenido generado por la IA.
- Debes buscar siempre la revisión de otras personas, no te limites a confiar en tu propia validación.

Resultados de esta etapa:

- Desarrollar habilidades de prompting tanto generales como de ámbito especifico.
- Aumentar tu productividad con un uso eficaz de las herramientas de IA.
- Seguir fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Ten en cuenta que la IA puede no tener la respuesta correcta porque esa respuesta no esté ni siquiera en Internet. Además, si te da soluciones incorrectas, intenta no insistir y busca ayuda entre las personas que te rodean. Vas a ahorrarte tiempo y vas a sumar en compresión.
- Vas a enfretarte con frecuencia a situaciones (como exámenes o evaluaciones) donde debes demostrar una comprensión real. Prepárate, sigue construyendo tanto tus habilidades técnicas como transversales.
- Explicar tu razonamiento y debatir con otras personas suele revelar lagunas en tu comprensión de un concepto. Prioriza el aprendizaje entre pares.
- Lo normal es que la herramienta de IA que utilices no conozca tu contexto específico (a menos que se lo indiques), así que te dará respuestas genéricas. Si buscas información más adecuada y más precisa en relación a tu entorno cercano, confía en el resto de estudiantes.
- Donde la IA tiende a generar la respuesta más probable, el resto de estudiantes puede proporcionar perspectivas alternativas y matices valiosos. Confía en la comunidad de 42 como un punto de control de calidad.

✓ Buenas prácticas:

> Le pregunto a la IA: "¿Cómo pruebo una función de ordenación?"Me da algunas ideas. Las pruebo y reviso los resultados con otra persona. Refinamos el enfoque de manera conjunta.

✗ Mala práctica:

> Le pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad. Suspendo mi proyecto.

✓ Buenas prácticas:

> Utilizo la IA para ayudarme a diseñar un parser. Luego, reviso la lógica con otra persona. Encontramos dos errores y lo reescribimos juntos: mejor, más limpio y comprendiendo al 100

✗ Mala práctica:

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y suspendo mi proyecto.

Capítulo III

Introducción

III.1. ¿Qué es la llamada a función?

Los Large Language Models (LLMs) son muy potentes para entender y generar lenguaje humano, pero la respuesta no es una salida estructurada y ejecutable por una máquina. Los sistemas de llamada a función cubren esta brecha traduciendo las peticiones en lenguaje natural en llamadas a funciones precisas con argumentos tipados.

Por ejemplo:
Del lenguaje natural a la llamada a función

```
Peticion en lenguaje natural: "Cual es la suma de 40 y 2?"

Respuesta de un LLM normal: "La suma de 40 y 2 es 42."

Respuesta de un sistema de llamada a funcion:
{
"function": "add_numbers",
"arguments": {"a": 40, "b": 2}
}
```

El sistema de llamada a función no responde a la pregunta directamente. En su lugar, proporciona las **herramientas** para resolverla: el nombre correcto de la función y los argumentos adecuados con los tipos apropiados.

III.2. ¿Por qué es importante?

La llamada a función permite a los LLM:

- **Interactuar con sistemas externos**: Llamar a APIs, hacer consultas a bases de datos, controlar dispositivos, etc.
- **Ejecutar código**: Realizar cálculos, transformaciones de datos, y operaciones con archivos.
- **Encadenar operaciones**: Dividir tareas complejas en pasos ejecutables.
- **Proporcionar salida estructurada**: Generar archivos JSON, XML u otros formatos legibles por una máquina.
- **Extraer datos estructurados de texto no estructurado**: Por ejemplo, dado un libro extenso, extraer campos como `{protagonist name, protagonist sex, protagonist age}`

Esta tecnología está detrás de los asistentes de IA modernos, las herramientas de generación de código y los agentes autónomos. A la vez que permite tareas como la extracción automática de información y la estructuración de conocimiento a partir de texto en bruto.

III.3. El desafío

Los modelos de lenguaje pequeños (como el modelo de 0.6B parámetros que vas a usar) son poco fiables para generar una salida estructurada. Cuando se les pide que produzcan un JSON, tienen éxito cerca del 30 % de las veces. Sin embargo, los modelos en producción alcanzan una fiabilidad del 99 % o más con estos mismos modelos pequeños.

¿Cómo? La respuesta está en la **decodificación restringida**, una técnica que guía la salida del modelo token a token para garantizar una estructura válida, sin depender únicamente de los prompts.

Capítulo IV

Common Instructions

IV.1. General Rules

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- Your functions should handle exceptions gracefully to avoid crashes. Use `try-except` blocks to manage potential errors. Prefer context managers for resources like files or connections to ensure automatic cleanup. If your program crashes due to unhandled exceptions during the review, it will be considered non-functional.
- All resources (e.g., file handles, network connections) must be properly managed to prevent leaks. Use context managers where possible for automatic handling.
- Your code must include type hints for function parameters, return types, and variables where applicable (using the `typing` module). Use `mypy` for static type checking. All functions must pass mypy without errors.
- Include docstrings in functions and classes following PEP 257 (e.g., Google or NumPy style) to document purpose, parameters, and returns.

IV.2. Makefile

Include a `Makefile` in your project to automate common tasks. It must contain the following rules (mandatory lint implies the specified flags; it is strongly recommended to try `-strict` for enhanced checking):

- **install**: Install project dependencies using `pip`, `uv`, `pipx`, or any other package manager of your choice.
- **run**: Execute the main script of your project (e.g., via Python interpreter).
- **debug**: Run the main script in debug mode using Python's built-in debugger (e.g., pdb).
- **clean**: Remove temporary files or caches (e.g., __pycache__, .mypy_cache) to keep the project environment clean.

- **lint**: Execute the commands `flake8 .` and `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
- **lint-strict** (optional): Execute the commands `flake8 .` and `mypy . --strict`

IV.3. Additional Guidelines

- Create test programs to verify project functionality (not submitted or graded). Use frameworks like `pytest` or `unittest` for unit tests, covering edge cases.
- Include a `.gitignore` file to exclude Python artifacts.
- It is recommended to use virtual environments (e.g., venv or conda) for dependency isolation during development.

If any additional project-specific requirements apply, they will be stated immediately below this section.

IV.3.1. Requisitos adicionales

- Todas las clases deben usar **pydantic** para validación.
- Puedes usar los paquetes **numpy** y **json**.
- El uso de **dspy** (o cualquier paquete similar) está completamente prohibido, incluyendo pytorch, el paquete de huggingface, transformers, outlines, etc.
- Tienes que usar los siguientes modelos:
  - **Qwen/Qwen3-0.6B** (por defecto)
  - Puedes usar otros modelos siempre que tu proyecto funcione con **Qwen/Qwen3-0.6B**.
- La función a llamar debe elegirse usando el LLM, no con heurísticas ni ningún otro tipo de magia medieval.
- Está prohibido usar cualquier método o atributo privado del paquete `llm_sdk`.
- Debes crear un entorno virtual e instalar los paquetes **numpy** y **pydantic** usando **uv**. Para usar **llm_sdk**, puedes copiarlo en el mismo directorio en el que está `src`.
- Las personas evaluadoras y la moulinette solo ejecutarán `uv sync` para configurar el entorno.
- Todos los errores deben gestionarse correctamente. Tu programa nunca debe fallar de forma inesperada y siempre debe proporcionar mensajes de error claros a la persona usuaria.

IV.3.2. Uso

Tu programa debe ejecutarse usando el siguiente comando (donde `src` es la carpeta que contiene tus archivos):
Ejecución del programa

```
uv run python -m src [--input <input_file>] [--output <output_file>]
```



Por defecto, el programa leerá los archivos de entrada del directorio data/input/ y escribirá la salida en el directorio data/output/. Opcionalmente puedes especificar rutas personalizadas usando los argumentos -input y -output. Por ejemplo:

```
uv run python -m src --input data/input/example.json --output data/output/function_calling_results.json
```

Capítulo V

Parte obligatoria

V.1. Resumen

En este proyecto, crearás una herramienta de llamada a función que traduzca peticiones en lenguaje natural en llamadas a funciones estructuradas. Dada una pregunta como "What is the sum of 40 and 2?", tu solución no debe devolver 42, sino proporcionar:

- El nombre de la función: `fn_add_numbers`
- Los argumentos: `.ª": 40, "b": 2`

Tu implementación debe usar **decodificación restringida** para garantizar un JSON válido al 100 %, asegurando una fiabilidad casi perfecta incluso con un modelo pequeño de 0.5B parámetros.

V.2. Archivos de entrada

Tu solución procesará los dos archivos de entrada situados en el directorio `data/input/`:

V.2.1. Pruebas de llamadas a funciones

El archivo `data/input/function_calling_tests.json` contiene un array JSON de prompts en lenguaje natural que tu programa debe procesar.

Ejemplo: function_calling_tests.json

```
[
    "What is the sum of 2 and 3?",
    "Reverse the string 'hello'",
    "Calculate the factorial of 5"
]
```

V.2.2. Definiciones de funciones

El archivo `data/input/function_definitions.json` contiene las funciones disponibles que tu sistema puede llamar. Cada función incluye:

- Nombre de la función
- Nombres de los argumentos y tipos
- Tipo de retorno
- Descripción

Ejemplo: function_definitions.json

```
[
    {
        name": "fn_add_numbers",
        "description": "Add two numbers",
        "parameters": {
            "a": {"type": "number"},
            "b": {"type": "number"}
        },
        "returns": {"type": "number"}
    },
    {
        "name": "fn_reverse_string",
        "description": "Reverse a string",
        "parameters": {
            "s": {"type": "string"}
        },
        "returns": {"type": "string"}
    }
]
```



Estos ejemplos sirven como referencia del nivel de complejidad esperado. Sin embargo, tu solución se probará con distintos prompts y conjuntos de funciones. Debes implementar una gestión adecuada de errores de JSON para los archivos de entrada, ya que pueden contener JSON no válidos o directamente no existir.

V.3. Interacción con el LLM

V.3.1. El SDK del LLM

Adjunto a este proyecto encontrarás el wrapper **Small_LLM_Model** en el paquete `llm_sdk` que puedes usar para interactuar con el LLM.

El SDK proporciona varios métodos esenciales:

- `get_logits_from_input_ids(input_ids: Tensor) ->Tensor`
  Recibe un tensor `input_ids` y devuelve los logits en bruto tras llamar al modelo LLM.
- `get_path_to_vocabulary_json() ->str`
  Devuelve la ruta a un archivo JSON que contiene la correspondencia estructurada entre `input_ids` y tokens.
- `encode(text: str) ->List[int]`
  Codifica una cadena de texto en su lista correspondiente de IDs de tokens usando el tokenizador del modelo.
- `decode(token_ids: List[int]) ->str` *(opcional)*
  Opcionalmente decodifica una lista de IDs de tokens de vuelta a una cadena de texto.

V.3.2. El proceso de generación

El proceso de generación del LLM sigue estos pasos:

1. **Prompt**: Tu pregunta en lenguaje natural
   Ejemplo: *"What is the sum of 2 and 3?"*
2. **Tokenización**: El texto se divide en unidades de subpalabras (tokens). A diferencia de una simple división por palabras, los tokenizadores suelen incluir espacios, dividir palabras en unidades más pequeñas y manejar la puntuación de forma compleja.
   Ejemplo: *["What", ïs", "the", "sum", .ºf", , "2", .ªnd", , "3", ¿"]*
   Nota: Esta es una representación simplificada. Los tokenizadores reales usan algoritmos como BPE o WordPiece, y la tokenización exacta depende del vocabulario del modelo.
3. **Input IDs**: Los tokens se convierten en IDs numéricos que el modelo entiende.
   Ejemplo (ilustrativo): *[892, 318, 262, 4771, 286, 16, 290, 17, 30]*
4. **Procesamiento en el LLM**: El modelo procesa estos números a través de su red neuronal.
5. **Logits**: El modelo genera puntuaciones de probabilidad para cada posible siguiente token.
   Ejemplo: *token_5: 0.001, token_42: 0.85, token_100: 0.02, ...*

6. **Selección de tokens**: El siguiente token se elige en función de estas probabilidades, normalmente el que tiene la puntuación más alta.
   En esta etapa se pueden aplicar técnicas como la ***decodificación restringida*** *para limitar las opciones de tokens y asegurar que la salida siga una estructura específica, como generar un JSON válido al 100 %.*

Importante: Este proceso se repite token a token. Cada token generado se añade al prompt y los pasos 2-6 se repiten hasta que se genera la respuesta completa.

Vista simplificada:

Proceso de generación del LLM:

```
Prompt -> Tokenization -> Input IDs -> LLM -> Logits -> Next Token Selection
```

V.3.3. Comprender la decodificación restringida

Los modelos de lenguaje generan texto añadiendo token tras token. En cada paso, el modelo produce una distribución de probabilidad (logits) sobre todos los posibles tokens siguientes. Normalmente, se muestrea a partir de esta distribución o se elige el token con mayor probabilidad.

La **decodificación restringida** interviene en este proceso modificando los logits *antes* de la selección del token:

1. El modelo produce logits para todos los tokens posibles.
2. Identificas qué tokens mantendrían tanto una estructura JSON válida *como el cumplimiento del esquema esperado.*
3. Estableces los logits de los tokens no válidos (los que rompen el esquema o la estructura) a menos infinito.
4. Muestras solo los tokens restantes válidos.

En este proyecto, la decodificación restringida no solo debe asegurar un JSON sintácticamente válido, sino también imponer un esquema específico. Por ejemplo, si el campo del JSON `"firmware"` solo puede tomar unos pocos valores predefinidos, el decodificador debería restringir la selección de tokens a esas opciones permitidas. Esto garantiza que cada token generado mantenga la validez tanto estructural como semántica, imponiendo el esquema requerido. Como resultado, el JSON producido es 100 % recuperable y siempre se puede parsear sin errores.



Tu solución NO debe basarse en que el modelo produzca espontáneamente un JSON correcto a partir de un prompt. Pedir al modelo las definiciones de funciones y esperar salida estructurada no es fiable, y no es la habilidad que esperamos que desarrolles en este proyecto.



Piensa en cómo puedes usar el archivo de vocabulario JSON para mapear entre tokens y sus representaciones en forma de cadena. Esto es crucial para determinar qué tokens son válidos en cada paso de generación.

V.4. Formato del archivo de salida

Tu programa producirá un único archivo JSON: `output/function_calling_results.json`. Para cada prompt, añade un objeto JSON a este archivo. Cada objeto del array debe contener exactamente las siguientes claves:

- `prompt` (string): La petición original en lenguaje natural
- `fn_name` (string): El nombre de la función a llamar
- `args` (object): Todos los argumentos requeridos con los tipos correctos

V.4.1. Ejemplo de salida

data/output/function_calling_results.json

```
[
    {
        "prompt": "What is the sum of 2 and 3?",
        "fn_name": "fn_add_numbers",
        "args": {"a": 2.0, "b": 3.0}
    },
    {
        "prompt": "Reverse the string 'hello'",
        "fn_name": "fn_reverse_string",
        "args": {"s": "hello"}
    }
]
```

V.4.2. Reglas de validación

- El archivo debe ser un JSON válido (sin comas finales, sin comentarios)
- Las claves y tipos deben coincidir exactamente con el esquema de `function_definitions.json`
- No se permiten claves adicionales ni texto libre en ninguna parte de la salida
- Todos los argumentos requeridos deben estar presentes
- Los tipos de los argumentos deben coincidir con los de la definición de la función (number, string, boolean, etc.)



Los archivos de entrada proporcionados pueden cambiar durante la evaluación. No hardcodees soluciones basadas en los ejemplos dados.

V.5. Rendimiento y fiabilidad

Tu implementación debería alcanzar:

- **Precisión casi perfecta**: Más del 95 % de selección correctade función y argumentos.
- **JSON válido al 100 %**: Toda la salida debe ser parseable y cumplir el esquema.
- **Velocidad razonable**: Procesar todos los prompts de prueba en menos de 5 minutos.
- **Gestión robusta de errores**: Manejar de forma correcta entradas mal formateadas, archivos ausentes y casos límite.



El modelo Qwen3-0.6B tiene solo 500 millones de parámetros y, aun así, con una decodificación restringida adecuada puede alcanzar una fiabilidad comparable a la de modelos mucho más grandes. Esto demuestra el poder de una guía estructural frente a la fuerza bruta del tamaño del modelo.

V.6. Probar tu implementación

Para verificar que tu solución funciona correctamente:

1. Asegúrate de que los archivos de entrada están en el directorio `input/`.
2. Ejecuta: `uv run python -m src`.
3. Comprueba que se ha creado `output/function_calling_results.json`.
4. Valida la estructura y el contenido del JSON.
5. Verifica que los nombres de las funciones y los tipos de los argumentos coinciden con las definiciones.



Haz pruebas con distintos casos límite: cadenas vacías, números grandes, caracteres especiales, prompts ambiguos y funciones con múltiples parámetros.

Capítulo VI

Requisitos del Readme

Debe incluirse un archivo `README.md` en la raíz del repositorio Git. Su propósito es permitir que cualquier persona que no esté familiarizada con el proyecto (pares, personal, responsables de selección, etc.) pueda entender rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.
El `README.md` debe incluir, como mínimo:

- La primera línea debe estar en cursiva y decir: *Este proyecto ha sido creado como parte del currículo de 42 por <login1>[, <login2>[, <login3>[...]]].*
- Una sección de "**Descripción**"que presente claramente el proyecto, incluyendo su objetivo y una breve visión general.
- Una sección de "**Instrucciones**"que contenga cualquier información relevante sobre compilación, instalación y/o ejecución.
- Una sección de "**Recursos**"que enumere referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción del uso de IA, especificando para qué tareas y en qué partes del proyecto se ha utilizado.

➠ **Podrían requerirse secciones adicionales dependiendo del proyecto** (por ejemplo, ejemplos de uso, lista de características, decisiones técnicas, etc.).

Cualquier contenido extra requerida se listará explícitamente a continuación.

Para este proyecto, el `README.md` también debe incluir:

- **Explicación del algoritmo**: Describe en detalle tu enfoque de decodificación restringida.
- **Decisiones de diseño**: Explica las elecciones clave en tu implementación.
- **Análisis de rendimiento**: Comenta la precisión, velocidad y fiabilidad de tu solución.
- **Retos encontrados**: Documenta las dificultades que has encontrado y cómo las has resuelto.

- **Estrategia de pruebas**: Explica cómo has validado tu implementación.
- **Ejemplos de uso**: Proporciona ejemplos claros de cómo ejecutar tu programa.



`La elección del idioma es a tu discreción. Se recomienda escribir en inglés, pero no es obligatorio.`

Capítulo VII

Entrega y evaluación entre pares

Entrega tu trabajo en tu repositorio `Git` como de costumbre. Solo se evaluará durante la defensa el trabajo que haya dentro de tu repositorio. No dudes en comprobar dos veces los nombres de tus archivos para asegurarte de que son correctos.

Tu repositorio debe contener:

- El directorio `src/` con tu implementación
- `pyproject.toml` y `uv.lock` para la gestión de dependencias
- El directorio `llm_sdk/` (copiado del paquete proporcionado)
- El directorio `data/input/` con archivos de prueba (para demostración)
- `README.md` con documentación completa
- Cualesquiera archivos adicionales necesarios para ejecutar tu solución



`No incluyas el directorio output/ en tu repositorio. Se generará durante la evaluación.`

VII.1. Instrucciones de recodificación

Durante la evaluación, es posible que se solicite una ligera **modificación del proyecto**. Esto puede consistir en ajustar ligeramente el comportamiento, modificar unas cuantas líneas de código o incorporar una característica fácil de implementar.

Puede que este paso **no sea necesario en todos los proyectos**, pero hay que tenerlo en cuenta si así se especifica en la hoja de evaluación.

Este paso sirve para verificar la comprensión real de una parte específica del proyecto. La modificación se puede realizar en cualquier entorno de desarrollo que se elija (por ejemplo, la configuración habitual), y debería ser factible en unos pocos minutos, a menos que se

defina un plazo específico como parte de la evaluación.
Por ejemplo, se puede pedir hacer una pequeña actualización en una función o *script*, modificar lo que se vería en pantalla o ajustar una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán cada **hoja de evaluación** y pueden variar de una evaluación a otra para el mismo proyecto.